CSE445 Machine Learning — Final Exam Prep

Decision Trees (Classification & Regression) + Random Forest

LECTURE 9: DECISION TREE (CLASSIFICATION)

1. What is a Decision Tree?

A **decision tree** is a **supervised learning algorithm** usable for **both classification and regression**. It mimics human decision-making by splitting data into branches at decision points (**nodes**) until it reaches a final prediction (a **leaf**). Think of it as a flowchart of yes/no (or condition-based) questions.

Classification Trees → predict a **category** (e.g., "spam" / "not spam")
Regression Trees → predict a **continuous value** (e.g., house price)

2. Key Terms / Components

Term	Meaning	Example
Root Node	The topmost decision point; splits using the *most informative* feature	"Income > $50k?"
Internal Nodes	Decision points after the root; keep splitting on feature conditions	"Credit Score > 700?"
Leaf Nodes	Final nodes — hold the predicted outcome; no further splitting	"Approve loan" / "Reject loan"

3. Applications

Healthcare – diagnosing disease from symptoms

Finance – credit risk assessment, fraud detection
Marketing – customer segmentation ("Will this user buy Product X?")
Manufacturing – quality control ("Is this product defective?")

4. How Splitting Works: Key Criteria (impurity measures)

The tree decides *how* to split a node using **impurity measures**. The goal is to **maximize homogeneity (purity)** in the resulting child nodes.

(a) Entropy — measures disorder in the data

$$Entropy = -\sum_i p_i \log_2 p_i$$

p_i = proportion (probability) of class i in the node.
Lower entropy → purer node. Entropy = 0 means the node has only one class. Entropy = 1 (for 2 classes) means a perfect 50/50 mix (maximum disorder).

(b) Gini Index — also measures disorder, but computed differently

$$Gini = 1 - \sum_i p_i^2$$

Ranges from **0 (pure node)** to **0.5 (maximally mixed, 2-class case)**.
Faster to compute than entropy (no logarithms) — this is why many practical implementations (e.g., sklearn default, CART) prefer Gini for speed.

(c) Information Gain (IG) — decides ***which feature*** to split on

$$IG = Entropy(parent) - WeightedEntropy(children)$$

The feature/split that gives the **highest IG** is chosen at each node.
Higher IG → better split (it reduces uncertainty the most).
(The Gini-based analogue is sometimes called **Gini Gain**, computed the same way but with Gini instead of Entropy.)

Exam tip: Entropy and Gini answer "how impure is this node?" IG answers "how much did splitting on this feature *reduce* that impurity?" — IG is what's actually used to *choose* the split.

5. Worked Example #1 — Fruit Classification (Entropy & Gini)

Setup: 10 fruits: 5 apples, 5 oranges. Features: **Color** (Red/Orange) and **Texture** (Smooth/Bumpy).

Root: $P(apple) = P(orange) = 0.5$

$$Entropy_{root} = -[0.5 \log_2 0.5 + 0.5 \log_2 0.5] = 1.0 \quad (\text{max disorder})$$

Data:

	Apples	Oranges	Total
Red	5	0	5
Orange	0	5	5

	Apples	Oranges	Total
Smooth	4	1	5
Bumpy	1	4	5

Split 1: By Color

Entropy(Red) = 0 (pure apples), Entropy(Orange) = 0 (pure oranges)
Weighted entropy $= \frac{5}{10}(0) + \frac{5}{10}(0) = 0.0$
IG(color) = 1.0 − 0.0 = 1.0

Split 2: By Texture

Entropy(Smooth) = $-[\frac{4}{5}\log_2\frac{4}{5} + \frac{1}{5}\log_2\frac{1}{5}] = 0.72$
Entropy(Bumpy) = 0.72 (symmetric)
Weighted entropy $= \frac{5}{10}(0.72) + \frac{5}{10}(0.72) = 0.72$
IG(texture) = 1.0 − 0.72 = 0.28

Conclusion: Color wins (IG = 1.0 > 0.28) — it perfectly separates apples and oranges.

Same story with Gini:

$Gini_{root} = 1 - [(0.5)^2 + (0.5)^2] = 0.5$
Gini Gain (color) $= 0.5 - 0.0 = 0.5$

Gini(Smooth) = Gini(Bumpy) = 0.32 → weighted Gini(texture) = 0.32 → Gini Gain(texture) $= 0.5 - 0.32 = 0.18$
Color wins again — both criteria agree.

6. Worked Example #2 — Iris Dataset (Mathematical Walkthrough)

Iris dataset: 3 classes (Setosa, Versicolor, Virginica), 50 instances each (150 total). Setosa is linearly separable from the other two; Versicolor and Virginica are **not** linearly separable from each other.

Step 1 – Root entropy (3 equally likely classes, $p_i = 1/3$ each):

$$Entropy_{root} = -3\left[\frac{1}{3}\log_2\frac{1}{3}\right] = 1.585$$

Step 2 – Candidate split: "Petal Length ⩽ 2.45 cm"

Child	Samples	Setosa	Versicolor	Virginica
Left (⩽2.45)	50	50	0	0
Right (>2.45)	100	0	50	50

Step 3 – Child entropies:

$Entropy_{left} = 0.0$ (pure Setosa)
$Entropy_{right} = -[\frac{50}{100}\log_2\frac{50}{100} + \frac{50}{100}\log_2\frac{50}{100}] = 1.0$

Step 4 – Weighted entropy:

$$Entropy_{weighted} = \frac{50}{150}(0.0) + \frac{100}{150}(1.0) = 0.666$$

Step 5 – Information Gain:

$$IG = 1.585 - 0.666 = 0.919$$

Interpretation: This IG is very high (≈ 58% of parent entropy) → the split is highly effective. It perfectly isolates Setosa (100% pure left node) while leaving the harder Versicolor/Virginica separation for deeper splits.

7. Stopping Criteria (when does a tree stop growing?)

Node is pure (entropy = 0)
Maximum tree depth reached — prevents deep trees that overfit
IG below a minimum threshold — rejects splits that don't improve the model enough (user-set threshold, guards against overfitting)
Minimum samples per leaf — don't split if it results in too few samples in a leaf
Minimum samples per split — don't even consider splitting a node unless it has enough samples

8. Interpreting IG magnitude

$0 \le IG \le Entropy_{root}$
Min IG = 0: split gives no reduction in entropy (children as impure as parent)
Max IG = Entropy(root): split creates perfectly pure children
Since IG's absolute value depends on the parent's entropy (which varies by dataset), we judge it **relative to parent entropy**:
High IG: > 20–30% of parent entropy
Low IG: < 5–10% of parent entropy
Example: IG = 0.919 vs. parent entropy 1.585 → 58% → highly effective split
Relative to other splits: the tree always compares IG across *all* candidate splits at a node and picks the highest — absolute value matters less than **rank**.
Domain-specific stopping thresholds: minimum IG threshold (reject low-IG splits) and statistical significance (IG should be meaningfully above 0).

9. Decision Tree with sklearn (Iris)

```python
```

```
X = iris.iloc[:, :-1]
y = iris.iloc[:, 4]

X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.33, random_state=42)

treemodel = DecisionTreeClassifier(criterion='entropy', random_state=0)
treemodel.fit(X_train, y_train)

tree.plot_tree(treemodel, filled=True, rounded=True,
        feature_names=["SL","SW","PL","PW"])

ypred = treemodel.predict(X_test)
score = accuracy_score(ypred, y_test)  # Accuracy on test data: 98%
```

`criterion='entropy'` tells sklearn to use Information Gain (entropy-based) rather than the default Gini for splitting.
The Iris decision tree trained this way achieves **98% test accuracy**.

LECTURE 10: DECISION TREE (REGRESSION)

1. What is Regression Tree?

Decision Tree Regression models relationships by **recursively partitioning data** into subsets, forming a tree structure. Unlike classification trees (predict categories), **regression trees predict continuous numerical values** — each **leaf node holds a predicted numeric value** (typically an average).

2. Step-by-Step Algorithm

Select the best split — using **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

where $\bar{y}$ = mean of target values, y_i = actual value, n = number of samples in the node.

Choose the split that gives the **maximal MSE reduction** (i.e., parent MSE − weighted child MSE is largest).
Partition Data — split into left (⩽ threshold) and right (> threshold) subsets.
Recurse — repeat the process on child nodes until a stopping criterion is met.
Assign Leaf Value — at each leaf, predict the **average of the target values** in that leaf:

$$Prediction = \frac{1}{n_{leaf}} \sum_{i \in leaf} y_i$$

Exam tip: This is the regression-tree analogue of classification's Entropy/Gini + IG: **MSE plays the role of the impurity measure**, and **"MSE Reduction" plays the role of Information Gain** — pick the feature/split with the highest MSE reduction.

3. Stopping Criteria

Node is pure (all identical target values)
Maximum tree depth reached (hard stop — prevents overfitting)
Minimum MSE reduction — stop splitting if MSE reduction < threshold
Minimum samples per leaf
Minimum samples per split

4. Worked Example — Predicting Hours Played (Cricket Match)

Scenario: A sport-hosting company wants to decide whether to host a cricket match, using weather data (Outlook, Temperature, Humidity, Wind) to predict **Hours Played**.

Step 0 — Root MSE ($n = 14$):

$$\bar{y} = \frac{25 + 30 + \cdots + 44 + 30}{14} = 39.79, \qquad MSE_{hour} = 86.88$$

Iteration 1 (root split)

Category	Best Subcategory	Weighted MSE	Weighted MSE Reduction
Outlook	Overcast vs. Rainy/Sunny	67.31	**19.57** ✅ (highest)
Temperature	Cool/Hot/Mild	79.58	7.30
Humidity	High/Normal	81.98	4.90

Category	Best Subcategory	Weighted MSE	Weighted MSE Reduction
Wind	False/True	83.51	3.37

Outlook gives the largest MSE reduction → **root split = "Outlook == Overcast?"** (Note: within Outlook, "Overcast" alone is a **pure/near-pure leaf**, MSE=12.19, reduction 74.69 — this is why Outlook dominates.)

Iteration 2a: Outlook ≠ Overcast branch (n=10, mean=37.2, MSE=93.36)

Category	Best subcategory split	Weighted MSE	Weighted MSE Reduction
Outlook (Rainy/Sunny)		89.36	**4.00** ✅ (highest)
Temperature		68.00	25.36
Humidity		75.72	17.64
Wind		80.16	13.20

Wait — actually check carefully: **Temperature gives the highest reduction (25.36)** in this branch, so it becomes the next split (matches the drawn tree, where "Temperature Hot" is the 2nd-level node under "No"/≠Overcast).

Iteration 2b: Outlook == Overcast branch (n=4, mean=46.25, MSE=12.19)

Category	Weighted MSE	Weighted MSE Reduction
Temperature	0.50	**11.69** ✅ (highest)
Humidity	4.62	7.56
Wind	10.62	1.56

Temperature wins here too → next split under the Overcast branch.

Final Tree (after 4 iterations)

```
Outlook == Overcast? (n=14, avg=39.79)
├── No → Temperature == Hot? (n=10, avg=37.2)
│       ├── No → Wind == False? (n=8, avg=39.625)
│       │       ├── No → Outlook==Sunny? (n=3, avg=33.67) → No: avg=26.5 | Yes: avg=48.0
│       │       └── Yes → Outlook==Sunny? (n=5, avg=43.2) → No: avg=47.67 | Yes: avg=36.5
│       └── Yes → Wind == True? (n=2, avg=27.25) → No: avg=25.0 | Yes: avg=30.0
└── Yes → Temperature == Mild? (n=4, avg=46.25)
        ├── No → Humidity==Normal? (n=3, avg=44.33) → No: avg=46.0 | Yes:
Temp==Cool?(n=2,avg=43.5)→No:44.0|Yes:43.0
        └── Yes → n=1, avg=52.0 (leaf)
```

Prediction Example 1: Outlook=Rainy, Temperature=Mild, Wind=False, Humidity=Normal

Outlook≠Overcast (No) → Temperature≠Hot (No) → Wind=False (Yes) → Outlook=Sunny? → No (it's Rainy) → **Leaf: avg = 47.67 hours**

Prediction Example 2: Outlook=Rainy, Temperature=Hot, Wind=True, Humidity=Normal

Outlook≠Overcast (No) → Temperature=Hot (Yes) → Wind=True → **Leaf: avg = 25.0 hours**

Exam tip: To predict with a regression tree, just walk root→leaf following the feature conditions, and the leaf's **stored average** is the prediction.

LECTURE 11: RANDOM FOREST

1. What is Random Forest?

A **Random Forest** is an **ensemble learning method** (supervised) for classification and regression. It builds **multiple decision trees** and merges their outputs for more accurate, stable predictions.

Random Forest = many weak learners (trees) → one strong model.

Outcome: Lower overfitting + Higher accuracy + Greater robustness.

2. Why Random Forest instead of a single Decision Tree?

✅ **Reduces overfitting** — randomness in data & features reduces variance; averaging many trees improves generalization
✅ **Improves accuracy** — aggregating many trees' predictions is more accurate/stable than one tree
✅ **Handles noisy data better** — robust to outliers/noise due to the ensemble effect

✅ **Works well with high-dimensional data** — random feature selection at each split helps even when many features are irrelevant
✅ **Less sensitive to training-data fluctuations** — a single decision tree can change drastically with small data changes; RF averages this out
✅ **Handles missing data** — can still make accurate predictions with incomplete data

3. Limitations of Random Forest

Slower to predict than a single decision tree (must query many trees)
Less interpretable than a single tree (can't easily "read" the logic)
May not perform well on sparse / very high-dimensional data

4. Ensembling — the general idea

Uses **multiple small (weak) models** instead of one complex model.
Each model alone isn't highly accurate, but **combining** results produces a stronger, more reliable prediction (like averaging opinions from a group instead of trusting one person).
Two main ensemble types:
Bagging (Bootstrap Aggregating): reduces **variance** by training models independently on random subsets of data (in parallel).
Boosting: reduces **bias** by sequentially improving models (each new model fixes the previous one's errors).
Random Forest = Bagging + Decision Trees.

5. How Random Forest Works (step by step)

Bootstrap Sampling — create N random subsets of the training data, **sampling with replacement** (same size as original). This ensures diversity and reduces class imbalance.
Base Model Training — train an independent decision tree on each bootstrapped subset. Trees train **in parallel** (saves time); different base learners can even be used for extra robustness.
Prediction Aggregation — combine all trees' outputs:
Classification: majority voting
Regression: averaging
Out-of-Bag (OOB) Evaluation — use the samples *not* selected in a tree's bootstrap sample to estimate that tree's/forest's performance.
Final Prediction — the aggregated output from all trees.

6. Worked Example — Heart Disease Random Forest

Original dataset (4 samples):

ChestPain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Step 1 — Bootstrapped Dataset

Randomly sample **with replacement**, same size as original. E.g., a bootstrapped set might duplicate the (Yes, No, Yes, 167, Yes) row twice and omit the (Yes, Yes, No, 210, No) row entirely. **Sampling with replacement means the same row can be picked more than once, and some original rows may not appear at all** (these become "out-of-bag" for that tree).

Step 2 — Build a Decision Tree on the bootstrapped data

Unlike a normal decision tree (which considers *all* features at each node), Random Forest **only considers a random subset of features at each split**. E.g., out of 4 features, randomly consider only feature 2 & 3 for the root node's split decision. This randomness in *feature selection* (on top of randomness in *data selection*) is what makes each tree different — this is the core trick behind Random Forest's diversity.

Step 3 — Repeat Steps 1-2 many times

Create a new bootstrapped dataset + a new random-feature tree, over and over (ideally **hundreds of times**). This produces a **variety of different trees**, which is what makes the forest effective (diverse trees make diverse — and less correlated — errors, which cancel out on averaging).

Making a Prediction (Aggregation)

New patient: ChestPain=Yes, Good Blood Circ.=No, Blocked Arteries=No, Weight=168, Heart Disease=? Run this sample down **every tree** in the forest and tally votes:

Heart Disease	Yes	No
Votes	3	1

Majority vote → "Yes" (predicted to have heart disease). (For regression forests, you'd **average** the leaf predictions instead of voting.)

Out-of-Bag (OOB) Evaluation

Each tree wasn't trained on *every* original row — the leftover rows (not picked during its bootstrap sampling) are that tree's **Out-of-Bag (OOB) data**.
Run each OOB sample through the trees that **didn't** see it during training, and check whether the (aggregated) prediction matches the true label.
Repeat for all OOB samples; tally correct vs. incorrect.
OOB Error = proportion of OOB samples predicted **incorrectly** → gives a built-in estimate of model accuracy **without needing a separate validation/test set**.

Finding the "Best" Model

Build a random forest.
Check its (OOB) accuracy.
Change the number of features considered at each split (e.g., 2 → 3) and rebuild.
Repeat several times, and keep the random forest with the best accuracy.

Exam tip: The **only** hyperparameter being tuned in this specific example is "number of features considered per split" — but in general you could also tune number of trees, tree depth, etc.

7. Missing Data — Types

Type	Meaning	Example	How to handle
Structurally Missing	Data that isn't meant to exist	Age of a child when respondent has no children	Best handled by **excluding** such observations
MCAR (Missing Completely At Random)	No pattern/reason at all	Machine failure randomly wipes some data	Safe to ignore/delete without bias
MAR (Missing At Random)	Missingness relates to *other observed* variables	Missing height can be inferred from age, gender, etc.	**Can be imputed** using statistical techniques
NMAR (Not Missing At Random)	Missing due to unobserved/intentional reasons	Respondents skip sensitive survey questions	Hardest to handle — imputation may introduce bias

Imputation = assigning a value to a missing entry by inference from the values of the processes it's involved in.

8. Imputation using Random Forest

Why RF for imputation?

Powerful & efficient; scales well to large datasets
Handles **non-linear** relationships and **outliers**
Supports both **numerical and categorical** data
Includes **built-in feature selection**
Often outperforms KNN and other imputation techniques

Worked Example — Filling In a Missing Value

Dataset (4th row has missing Blocked Arteries and Weight):

ChestPain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	**???**	**???**	No

General idea: Make an initial (possibly bad) guess, then **iteratively refine** it using the random forest's proximity/similarity structure until the guess converges.

Step A — Initial Guess

Categorical (Blocked Arteries): use the **most common value among rows sharing the same class label** (Heart Disease = No) → "No" appears in 2/2 such rows → initial guess = **No**.
Numeric (Weight): use the **median** of that same class-label subgroup → median(125, 210) = **167.5**.

Step B — Build a Random Forest (say, 10 trees) using the dataset with the filled-in guesses.

Step C — Run all samples down all trees & build a Proximity Matrix

For each tree, note which samples land in the **same leaf node** — they're considered "similar."
Every time two samples share a leaf in a tree, increment their proximity-matrix cell by 1.
Example after Tree 1: only samples 3 & 4 share a leaf → matrix has a 1 in position (3,4)/(4,3).

After running through **all 10 trees**, sum up all the co-occurrences, e.g., sample 3 & 4 end up together in 8 of the 10 trees → raw proximity count = 8.
Normalize: divide every cell by the total number of trees (10) → e.g., proximity(3,4) = 8/10 = **0.8** (very similar), while proximity(1,2) might be 2/10 = 0.2 (less similar).

Step D — Use proximities to update/refine the missing values

For **categorical** features: take a **weighted average (proximity-weighted vote)** across the other samples' values for that feature, using proximity as the weight. E.g., "Blocked Arteries: Yes → 1/3 weight, No → 2/3 weight" → since No has higher weighted vote, guess stays/updates to **No**.
For **numerical** features: take a **proximity-weighted average** of other samples' values for Weight → refined guess, e.g., **198.5** (updated from the initial 167.5).

Step E — Repeat until convergence

Rebuild the forest on the revised dataset, recompute proximities, recompute the missing-value estimates, and repeat until the values **stop changing** between iterations (convergence).

9. Missing Data in a *New* (unseen) Sample

Suppose the Random Forest (10 trees) is already trained. A new patient arrives with a **missing feature AND unknown label**:

ChestPain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	???	168	???

Step 1 — Create two copies, one for each possible label:

Copy #1: label = "Yes" (has heart disease)
Copy #2: label = "No" (does not have heart disease)

Step 2 — Impute the missing feature separately in each copy (using the proximity-based iterative method from Section 8), conditioned on that copy's assumed label:

Copy #1 (assuming Yes) → Blocked Arteries imputed as **Yes**
Copy #2 (assuming No) → Blocked Arteries imputed as **No**

Step 3 — Run both completed copies through every tree in the forest and count how many trees "agree with"/correctly classify each copy:

Updated Copy #1 (Yes): correctly labeled by **7 of 10 trees**
Updated Copy #2 (No): correctly labeled by **3 of 10 trees**

Step 4 — Vote: Whichever copy gets **more agreeing trees wins**. Here Copy #1 (7 > 3) wins.

Result: the missing feature is filled in as **Blocked Arteries = Yes**, AND the sample is classified as **Heart Disease = Yes** — both the imputation and the classification are solved simultaneously by this procedure.

PROBABLE EXAM QUESTIONS & ANSWERS

A. Conceptual / Definition-style

Q1. What is a decision tree, and how does it differ for classification vs. regression? A: A decision tree is a supervised algorithm that splits data into branches at nodes based on feature conditions, ending in leaves with predictions. Classification trees predict discrete categories (leaf = class label, chosen e.g. by majority class); regression trees predict continuous values (leaf = average of target values in that leaf).

Q2. Define Root Node, Internal Node, and Leaf Node. A: Root Node — topmost node, splits on the most informative feature. Internal Nodes — intermediate decision points that further split the data. Leaf Nodes — terminal nodes holding the final predicted outcome; no further splitting.

Q3. What is entropy? What do high and low entropy mean? A: Entropy $= -\sum p_i \log_2 p_i$ measures disorder/impurity in a node. High entropy (up to 1 for 2 classes) = maximally mixed/uncertain node; low entropy (0) = pure node containing only one class.

Q4. What is the Gini Index, and how does it compare to entropy? A: Gini $= 1 - \sum p_i^2$ also measures impurity, ranging 0 (pure) to 0.5 (max mixed, 2 classes). It measures essentially the same thing as entropy but is **computationally cheaper** (no logs), so it's often preferred in practice (e.g., default in many implementations).

Q5. What is Information Gain, and how is it used? A: IG = Entropy(parent) − Weighted Entropy(children). It quantifies how much a candidate split reduces impurity. At each node, the tree evaluates IG for every candidate feature/split and picks the one with the **highest IG**.

Q6. What is the range of Information Gain, and what do the extremes mean? A: $0 \le IG \le Entropy_{root}$. IG = 0 means the split didn't reduce impurity at all (children as impure as the parent). IG = Entropy(root) (maximum) means the split produced perfectly pure children.

Q7. List the stopping criteria for a classification decision tree. A: (1) Node is pure (entropy=0), (2) maximum tree depth reached, (3) IG below a minimum threshold, (4) minimum samples per leaf not met, (5) minimum samples per split not met.

Q8. Why do we evaluate IG "relative to parent entropy" rather than as an absolute number? A: Because IG's magnitude depends on the parent node's entropy, which varies across datasets/splits — the same raw IG value could be "great" for one split and "poor" for another. So we express it as a % of parent entropy (e.g., >20–30% = high, <5–10% = low) for a fair comparison.

Q9. What criterion do regression trees use instead of entropy/Gini, and how are leaf predictions made? A: Regression trees use **Mean Squared Error (MSE)** as the impurity/error measure; splits are chosen to maximize **MSE reduction**. Each leaf's prediction is the **average (mean)** of the target values of the training samples that fall into it.

Q10. What is a Random Forest? A: An ensemble supervised-learning method that builds many decision trees (via bagging — bootstrap sampling of data + random feature subsets at each split) and aggregates their predictions (majority vote for classification, averaging for regression) to get more accurate, stable, and less overfit predictions than a single tree.

Q11. Why is Random Forest generally better than a single Decision Tree? A: It reduces overfitting/variance (via averaging many diverse trees), improves accuracy and stability, handles noisy data and high-dimensional data better, is less sensitive to small changes in training data, and can handle missing data.

Q12. What are the limitations of Random Forest? A: Slower prediction time than a single tree (must query many trees), less interpretable ("black box" vs. a readable single tree), and may not perform as well on very sparse/high-dimensional data.

Q13. Distinguish Bagging from Boosting. A: **Bagging** (Bootstrap Aggregating) trains models **independently/in parallel** on random data subsets to **reduce variance** (e.g., Random Forest). **Boosting** trains models **sequentially**, each one correcting the previous one's errors, to **reduce bias** (e.g., AdaBoost, Gradient Boosting).

Q14. Describe the main steps of how a Random Forest works. A: (1) Bootstrap sampling — create N random subsets (with replacement) from the training data; (2) train an independent tree on each subset, considering only a random subset of features at each split; (3) aggregate predictions across trees (majority vote/averaging); (4) use out-of-bag samples to evaluate performance; (5) output the final aggregated prediction.

Q15. What is "bootstrap sampling," and why is it done with replacement? A: Randomly drawing samples from the original dataset **with replacement** to build a new dataset of the same size. "With replacement" means a sample can be picked multiple times (and

some original samples may be left out entirely) — this introduces the diversity across trees that makes the ensemble effective.

Q16. Why does Random Forest randomly restrict the features considered at each split (in addition to bootstrapping the data)? A: To further **decorrelate** the trees — if all trees could always pick the single strongest feature, they'd end up very similar (correlated errors that don't cancel out when averaged). Restricting the feature subset forces trees to be more diverse, which improves the ensemble's variance-reduction benefit.

Q17. What is Out-of-Bag (OOB) error, and why is it useful? A: For each tree, the samples *not* selected in its bootstrap sample are its OOB samples. Running these through the tree(s) that didn't train on them and checking prediction accuracy gives the **OOB error** (proportion misclassified) — a built-in, "free" estimate of model accuracy without needing a separate held-out test/validation set.

Q18. What are the four types of missing data, and how do they differ? A: **Structurally missing** — value can't logically exist (exclude such rows). **MCAR** — missing with absolutely no pattern (e.g., random machine failure). **MAR** — missingness correlates with other observed variables (e.g., missing height inferable from age/gender) — can be imputed with statistical techniques. **NMAR** — missing for unobserved/intentional reasons (e.g., people avoiding a sensitive survey question) — hardest to handle, imputation may be biased.

Q19. Why is Random Forest well-suited for imputing missing data? A: It scales to large data, handles non-linear relationships and outliers, works with both numerical & categorical variables, has built-in feature selection, and often outperforms methods like KNN imputation.

Q20. Outline the Random Forest missing-value imputation procedure for a training-set example. A: (1) Make an initial guess (mode for categorical, median for numeric, computed within same-class rows); (2) build a random forest on the data with guesses filled in; (3) run all samples through all trees, and build a **proximity matrix** tracking how often pairs of samples land in the same leaf, normalized by the number of trees; (4) use these proximities as weights to recompute (refine) the missing values (weighted vote for categorical, weighted average for numeric); (5) repeat steps 2–4 until the imputed values **converge** (stop changing).

Q21. How does Random Forest impute AND classify a brand-new sample that has both a missing feature and an unknown label? A: Create two copies of the sample, one assuming each possible class label; impute the missing feature separately within each copy using the proximity method; run both completed copies through every tree in the forest and count how many trees "agree" (correctly classify) each copy; the label (and its

corresponding imputed value) belonging to the copy with more agreeing trees is chosen as the final answer — this simultaneously solves imputation and classification.

B. Broad / "Explain in detail" style

Q22. Walk through, step by step, how a decision tree decides the very first (root) split on a small dataset, using both entropy and Gini index. A: (Use the fruit example.) Compute the parent's entropy/Gini from the class proportions. For each candidate feature, split the data by that feature's values, compute the entropy/Gini of each resulting child, weight them by child-size proportion, and get a weighted child impurity. Subtract this from the parent's impurity to get Information Gain (or Gini Gain) for that feature. Repeat for all candidate features, and choose the feature with the **highest gain** as the root split. (Fully worked: Color gives IG=1.0/GG=0.5, beating Texture's IG=0.28/GG=0.18 — Color is chosen, and in fact perfectly separates the classes.)

Q23. Explain, with an example, how a regression tree decides where to split and what a leaf predicts. A: (Use the cricket-hours example.) At each node, for every candidate feature (and threshold, for numeric ones), split the data into two groups; compute each group's MSE around its own mean; compute the sample-size-weighted average MSE of the children; subtract from the parent's MSE to get the **MSE reduction**. Pick the feature/split with the largest MSE reduction. Recurse on children until a stopping rule is hit. Each final leaf's prediction is simply the **mean of the target (Hours Played) values** of the training rows that ended up there (e.g., a leaf with 3 samples averaging 47.67 hours predicts 47.67 for any new sample routed there).

Q24. Describe the complete process by which a Random Forest is built and used to classify a new heart-disease patient, referencing bagging, feature randomness, and voting. A: Random Forest = bagging applied to decision trees. First, generate many bootstrapped datasets (sampling the original data with replacement, same size each time). For each bootstrapped dataset, grow a decision tree, but restrict each split to a **randomly chosen subset of the available features** rather than considering all of them (this decorrelates the trees). Repeat this hundreds of times to get a forest of diverse trees. To classify a new patient, run their feature values down **every** tree in the forest to get each tree's individual prediction, then combine all predictions by **majority vote** (classification) or **averaging** (regression) to produce the forest's final output. (Worked example: 4 trees vote 3-Yes/1-No → final prediction = Yes.)

Q25. Explain the proximity-matrix method Random Forest uses for missing-value imputation, and why repeated iteration is necessary. A: After making an initial rough guess for missing values (mode/median within same-label rows), a random forest is trained, and every sample is passed down every tree. Whenever two samples land in the same leaf of a tree, that's evidence they're "similar," so their proximity-matrix entry is incremented. After running through all trees, each entry is normalized (divided by

number of trees) to give a similarity score between every pair of samples (0 = never together, 1 = always together). These proximities are then used as weights to produce an improved estimate of each missing value — similar samples' true values count more toward the estimate. Because a better-imputed dataset produces a different (hopefully better) forest, and a different forest produces different proximities, this whole process is **repeated iteratively**, refining the missing-value estimates each round, until they stabilize (converge) — i.e., no longer change meaningfully between iterations.

C. Short / "brief" style (rapid-fire)

Q: Formula for Entropy? A: $-\sum_i p_i \log_2 p_i$
Q: Formula for Gini Index? A: $1 - \sum_i p_i^2$
Q: Formula for Information Gain? A: $Entropy(parent) - WeightedEntropy(children)$
Q: Formula for MSE (regression tree)? A: $\frac{1}{n}\sum(y_i - \bar{y})^2$
Q: Regression tree leaf prediction formula? A: Average of target values in that leaf: $\frac{1}{n_{leaf}}\sum_{i \in leaf} y_i$
Q: Range of Gini Index (2-class)? A: 0 to 0.5
Q: Range of Entropy (2-class)? A: 0 to 1
Q: What does IG=0 mean? A: The split gave no improvement in purity.
Q: Random Forest uses which ensembling technique? A: Bagging (Bootstrap Aggregating).
Q: What does "bootstrap" mean here? A: Random sampling **with replacement** to build a new dataset the same size as the original.
Q: How does RF aggregate for classification vs regression? A: Majority voting vs. averaging.
Q: What is OOB data? A: The original samples *not* included in a tree's bootstrap sample — used to estimate error without a separate test set.
Q: Name the 4 types of missing data. A: Structurally missing, MCAR, MAR, NMAR.
Q: Which missing-data type is easiest to safely impute? A: MAR (Missing At Random) — related to other observed variables.
Q: What does a proximity matrix track? A: How often pairs of samples land in the same leaf node across all trees (a similarity score).
Q: One limitation of Random Forest? A: Less interpretable than a single decision tree (also: slower prediction, weaker on very sparse/high-dimensional data).

QUICK-REVIEW CHEAT SHEET

Concept	Classification Tree	Regression Tree
Predicts	Category/class	Continuous number
Impurity measure	Entropy or Gini Index	MSE
Split selection	Highest Information Gain (or Gini Gain)	Highest MSE Reduction
Leaf value	Majority class (or class probabilities)	Mean of target values in leaf

Concept	Formula
Entropy	$-\sum p_i \log_2 p_i$
Gini	$1 - \sum p_i^2$
Information Gain	$Entropy_{parent} - Entropy_{weighted,children}$
MSE	$\frac{1}{n}\sum(y_i - \bar{y})^2$
Regression leaf prediction	$\frac{1}{n_{leaf}} \sum_{i \in leaf} y_i$

Random Forest in one line: Bagging (bootstrap sampling + random feature subsets) applied to many decision trees, aggregated by voting/averaging, evaluated via OOB error, and also usable for missing-data imputation via proximity matrices.